

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-af1ed03dfc95e0cac.jsonl`
- SHA-256: `715c40a5725ffe2ca2432c01f562e3d3d1b748afd88eb041afa3b8e727b2b3d1`
- Source modified: `2026-06-21T14:05:19+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:00:37.402Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: cloud-one-click

CLAIMS:

```
[
  {
    "claim": "The compute backend is config-selected by ONE switch: get_compute_backend returns CloudRunCompute only for compute_target=='cloud_run', else None which runs the pipeline inline - the literal local<->cloud seam",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/compute/__init__.py:42-49"
  },
  {
    "claim": "The dispatched Cloud Run Job runs the SAME worker image forced INLINE (overrides compute_target=local_gpu + detector_impl=native) so the one artifact never re-dispatches - proving one image serves both roles",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/compute/cloud_run.py:39-44,103-132"
  },
  {
    "claim": "deployment.profile applies a coherent preset bundle (truly-local / cloud-serving / cpu-mock) over existing knobs; cloud-serving=cloud_run+native+gcs, truly-local=local_gpu+local; auto-detect SUGGESTS (K_SERVICE->cloud-serving, CI->cpu-mock, cuda->truly-local) but never auto-applies (D15)",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/presets.py (PROFILE_PRESETS, suggest_profile)"
  },
  {
    "claim": "The engine has zero app-layer auth and CORS allow_origins=['*'] and binds host=127.0.0.1 - so the EDGE gate (Cloudflare Access), not the app, is the tenancy boundary",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:51,735"
  },
  {
    "claim": "D9 mandates Cloudflare Access AND verifying the Cf-Access JWT signature AND locking Cloud Run ingress to the CF proxy, specifically so a direct hit to the Cloud Run URL cannot spoof the identity header",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-
```

indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md (D9, line 36)"

```
},
{
  "claim": "Cloudflare Access JWT is validated in Python with PyJWT against the team JWKS at
https://<team>.cloudflareaccess.com/cdn-cgi/access/certs checking aud + RS256; the signing key
rotates ~every 6 weeks so keys must be refreshed",
  "confidence": "high",
  "source": "https://developers.cloudflare.com/cloudflare-one/tutorials/fastapi/ +
https://developers.cloudflare.com/access/setting-up-access/validate-jwt-tokens"
},
{
  "claim": "Cloud Run NVIDIA L4 is ~$0.0001867/sec (~$0.67/hr), billed per-second, scales to
zero with no idle cost, needs >=4 CPU + >=16Gi, and reaches GPU-ready in ~5s with drivers pre-
installed",
  "confidence": "high",
  "source": "https://cloud.google.com/run/pricing +
https://docs.cloud.google.com/run/docs/configuring/jobs/gpu"
},
{
  "claim": "Render does NOT support GPU workloads and neither does Railway; among
Render/Railway/Fly only Fly offered GPUs - but Fly.io GPUs are DEPRECATED and unavailable after
Aug 1",
  "confidence": "high",
  "source": "https://render.com/articles/fastapi-deployment-options +
https://techsy.io/en/blog/railway-vs-render-vs-fly-io + https://fly.io/docs/gpus/"
},
{
  "claim": "'Deploy to Render' is a README button backed by a render.yaml Blueprint that
provisions on the USER's account on click; advise autoDeploy:false on the button blueprint",
  "confidence": "high",
  "source": "https://render.com/docs/deploy-to-render +
https://render.com/docs/infrastructure-as-code"
},
{
  "claim": "create_gpu_vm.sh is the single scripted BYO GPU path (L4 default, zone-sweeps
asia-south1->asia-southeast1, bakes Ops-Agent + rally-worker SA), and teardown.sh delete is the
cost-stop (a stopped VM still bills disk ~$34/mo)",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/gcp/create_gpu_vm.sh +
deploy/gcp/teardown.sh"
},
{
  "claim": "Per-job cost ledger (M8) computes cost_usd = sum(usage
gpu_seconds,wall_seconds,egress_gb x rate) + gemini_usd from a versioned pricebook, USD internal
/ INR display - but ships default-OFF ($0 placeholder pricebook) and worker usage-emission is a
follow-up",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/billing.py +
COMPUTE_DECOUPLED_SERVING/README.md (M8, line 123)"
},
{
  "claim": "WASB pretrained badminton weights have uncleared commercial-sharing provenance
(WASB-C3): NOT baked into the image, staged at runtime via $WASB_WEIGHTS, resolution required
before any public non-owner serving",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/deploy/cloudrun/Dockerfile:14-16,53-56"
},
{
  "claim": "Cloud-serving free-tier data-for-reels trade (D10): train ONLY on attested ADULT
footage + DERIVED data with raw auto-delete; minors->reel-yes-training-NO; paid tier=no-train;
counsel-gated to M9 - the data-leak/consent guardrail",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md (D10, line 37)"
}
```

```

    },
    {
      "claim": "India's DPDP Act (enforcement phase 2026) makes consent the primary lawful basis, requires deletion once business purpose ends, treats under-18 as children needing verifiable parental consent, and permits cross-border transfer except to restricted countries",
      "confidence": "medium",
      "source": "https://www.cookieyes.com/blog/india-digital-personal-data-protection-act-dpdpa/ + https://ksandk.com/data-protection-and-data-privacy/data-retention-and-deletion-under-indias-dpdp-rules/"
    },
    {
      "claim": "Twelve-factor build-once-deploy-many config-in-env is the standard pattern for one image serving local and cloud - swap a backing service via config only, no code change",
      "confidence": "high",
      "source": "https://aws.amazon.com/blogs/containers/developing-twelve-factor-apps-using-amazon-ecs-and-aws-fargate/"
    }
  ]

```

RECOMMENDATION UNDER TEST: Make ONE artifact, TWO official run modes, and resist hosted-PaaS GPU buttons. (1) THE SPINE - already built (M1-M8, default-OFF): ship one pip package (indexer console script) + one worker Docker image (deploy/cloudrun/Dockerfile), and let deployment.profile pick everything. backend/compute/___init___py:47-49 already makes local<->cloud a single config switch (cloud_run->CloudRunCompute, else inline); the dispatched Cloud Run Job runs the SAME image forced inline (cloud_run.py:41-44). Textbook twelve-factor; nothing new is needed to claim 'local AND cloud from one artifact' - only finishing M7 (real L4 run) and wiring worker usage-emission into the M8 ledger. (2) DOWNLOADABLE-APP 'push my workload to cloud' MODE: in the self-hosted UI Settings, surface profile as a toggle that calls suggest_profile() to PROPOSE a default but requires explicit confirm (D15 - cloud spend never silent). 'Push to cloud' = flip to cloud-serving (compute_target=cloud_run + storage=gcs); the local control plane then dispatches GPU detect+stitch to a Cloud Run Job and bucket-polls. Show the live per-video cost estimate (M8, INR) BEFORE confirming, and a hard per-video cap (open owner question in LOCAL_APPLIANCE_ARCHITECTURE section 10). (3) CLOUD TARGET = Cloud Run GPU Jobs, NOT Render/Railway/Fly for GPU: Render and Railway have no GPU; Fly's GPUs are deprecated after Aug 1. Cloud Run L4 (~\$0.67/hr, per-second, scale-to-zero, ~5s GPU-ready) is the only option that ALSO matches per-video billing and ADR-013 (GCP-only). Keep a 'Deploy to Render' button as a SECONDARY CPU-only control-plane distribution (Gemini/motion/stub modes that dispatch GPU work back to GCP) - lowest-friction one-click for a non-technical coach, but never the GPU path. (4) BYO-cloud: keep create_gpu_vm.sh/teardown.sh as the warm no-cold-start path for power users/owner; surface the always-DELETE rule in a UI teardown affordance. (5) NON-NEGOTIABLE before any public exposure: Cloudflare Access (cloudflared tunnel, no inbound ports) + PyJWT verification of the Cf-Access JWT signature (aud + JWKS refreshed for the ~6-week rotation) + Cloud Run ingress=internal-and-cloud-load-balancing - both header-trust AND signature-verify, because header-only trust is spoofable (D9). (6) NO DATA LEAK: truly-local needs no DPA; cloud-serving must honor D10 (adults-only training, derived-data, raw auto-delete, paid=no-train) and a retention sweep (originals are never deleted today) - all counsel-gated to M9 under DPDP/GDPR. Sequence: finish M7 (real L4 + DEPLOY_REPORT) -> land M9 edge-auth (default-OFF) -> wire ledger usage-emission + per-video cap -> add the UI profile toggle with cost-confirm -> optional Render control-plane button last.

2. assistant (2026-06-21T14:00:40.099Z)

I'll verify each load-bearing claim adversarially. Let me start by examining the actual code files referenced, then cross-check the external facts via web search.

3. user (2026-06-21T14:00:43.392Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/gcp-ops-agent-and-dryrun-guide'

from the remote, but no such ref was fetched.

---BRANCH---

feat/gcp-ops-agent-and-dryrun-guide

4. user (2026-06-21T14:00:43.399Z)

```

1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
8      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
9
10     """
11
12     from __future__ import annotations
13
14     import os
15     from typing import Any, Optional
16
17     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20     __all__ = [
21         "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
22     ]
23
24     def _storage_dict(config: Any) -> dict:
25         storage = getattr(config, "storage", None)
26         if storage is None:
27             return {}
28         if hasattr(storage, "model_dump"):
29             return storage.model_dump() # type: ignore[no-any-return]
30         return dict(storage) if isinstance(storage, dict) else {}
31
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46         """
47         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48         if target != "cloud_run":
49             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")

```

```

56     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57     client = run_client if run_client is not None else GoogleCloudRunJobClient()
58     return CloudRunCompute(
59         run_client=client, job_name=job_name, region=region, project=project,
60         storage_config=_storage_dict(config), policy=policy, token=token,
61     )
62

```

5. assistant (2026-06-21T14:00:46.650Z)

The branch differs from the snapshot ? I'm now on `feat/gcp-ops-agent-and-dryrun-guide`. Let me read the key files referenced in the claims.

6. user (2026-06-21T14:00:47.616Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU Job and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. Dispatch a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs>` (the SAME worker, on the L4
GPU).
7     2. Bucket-poll `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. Read the marker (it carries `video_id` + the worker's returncode) ? fetch
12        `outputs/<video_id>/report.json` ? `ComputeResult`.
13
14    The dispatched container is forced to run INLINE (`compute_target=local_gpu` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an injected client (`CloudRunJobClient`) so the whole
shim is unit-tested
18    with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
19    owner-verified on the M7 real run.
20    """
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
31
32    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34    from backend.storage import fetch, parse_uri
35    from backend.storage import get_storage
36
37    LOG = logging.getLogger("compute.cloud_run")
38
39    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch

```

```

branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name

```

```

92         self._region = region
93         self._project = project
94         self._storage_config = storage_config or {}
95         self._policy = policy
96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118         video_id = str(marker.get("video_id", ""))
119         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
120         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
121         rc = int(marker.get("returncode", 1))
122         if not video_id:
123             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
124                         "video_id) ? treating as failure", self._job_name)
125             rc = rc or 1
126         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
127         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
128         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
129         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
130
131     # --- storage-backed completion signal (bucket-poll) ---
132     def _read_marker(self, done_uri: str) -> Optional[dict]:
133         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
134         ref = parse_uri(done_uri)
135         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
136             return None
137         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
138
139     def _read_json(self, uri: str) -> dict:
140         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
141         # raises ValueError ? unlike local's identity passthrough. Without an explicit

```

```

dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

7. user (2026-06-21T14:00:47.687Z)

```

1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus

```



```

40 # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41 # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42 # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43 PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44     "truly-local": {
45         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46         # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47         # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48         "indexing": {"skip_ai_handoff": True},
49         "storage": {"backend": "local"},
50     },
51     "cloud-serving": {
52         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96

```

```

97     def probe_cuda_available() -> bool:
98         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
101         caller
102         that *wants* the GPU heads-up can pass the result in without the startup module
103         importing
104         torch itself. Best-effort by design: if torch lives only in a detector subprocess
105         env
106         (native WASB), this returns ``False`` in the main process ? which is why the startup
107         checks
108         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
109         hard
110         error that could false-fail."""
111         return _cuda_available()
112
113     def suggest_profile(
114         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
115     ) -> str:
116         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
117
118         Signals, in priority order:
119
120         - ``K_SERVICE`` set (Cloud Run)          ? ``"cloud-serving"``
121         - ``CI`` truthy                          ? ``"cpu-mock"``
122         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
123         - otherwise                             ? ``""`` (no strong suggestion)
124
125         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
126         never
127         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
128         passes
129         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
130         on it.
131
132         """
133         e = os.environ if env is None else env
134         if e.get("K_SERVICE"):
135             return "cloud-serving"
136         if _truthy(e.get("CI")):
137             return "cpu-mock"
138         if probe_cuda and _cuda_available():
139             return "truly-local"
140         return ""

```

8. assistant (2026-06-21T14:00:51.238Z)

Claims 1, 2, 3 are looking accurate so far. Let me verify the exact line numbers cited and check the main.py auth/CORS/host claims, plus the worker dispatch guard.

9. user (2026-06-21T14:00:52.018Z)

```

40     from backend.reporting import emit_indexer_report
41     from backend import storage # M2: URI-aware input acquisition (local = identity
42     passthrough)
43     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
44
45     app = FastAPI(title="Sports Highlight Indexer API")
46
47     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
48     is
49     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
50     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
51     app.add_middleware(

```

```

50     CORSMiddleware,
51     allow_origins=["*"],
52     allow_credentials=False,
53     allow_methods=["*"],
54     allow_headers=["*"],
55 )
56
57 # Serves static frontend files directly (now under api/ per approved structure)
58 os.makedirs("backend/api/static", exist_ok=True)
59 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
60
61 @app.get("/", response_class=HTMLResponse)
62 def serve_index():
63     index_path = "backend/api/static/index.html"
64     if os.path.exists(index_path):

```

10. user (2026-06-21T14:00:53.372Z)

```

720         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
721     )
722     if paths:
723         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
724         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
725         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
726     return
727
728 # Start Local Server Mode
729 if args.server or not args.video_path:
730     print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
731     # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
732     # in-process wake-timer for any parked 'waiting' job died with the old process,
so
733     # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
734     _get_executor().submit(_drain_due_jobs)
735     uvicorn.run(app, host="127.0.0.1", port=args.port)
736
737 if __name__ == "__main__":
738     # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
739     # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
740     # module object. If `main()` runs here it configures `__main__`'s globals, leaving
741     # `backend.main.db_instance` = None; the async-job drain then crashes silently on
742     # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
743     # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
744     # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
745     import backend.main as _bm
746
747     _bm.main()
748

```

11. assistant (2026-06-21T14:00:56.927Z)

Important finding on Claim 4. The code shows `allow\_credentials=False` and a comment explicitly stating that `allow\_origins=['\*']` with `allow\_credentials=True` is rejected. The claim says "CORS allow_origins=['*']" which is true, but the nuance is intentional. The host=127.0.0.1 bind at line 735 is confirmed. Let me check whether there's any app-layer auth (the `backend/api/auth.py` mentioned in git status).

12. user (2026-06-21T14:00:57.884Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

13. user (2026-06-21T14:00:58.139Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
```

```

42      && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43      hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45      # --- Main application env ---
46      WORKDIR /app
47      COPY pyproject.toml README.md ./
48      COPY backend ./backend
49      COPY training ./training
50      RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55      WASB_REPO_DIR=/opt/models/WASB-SBDT \
56      WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57      RALLY_DETECTOR_IMPL=native
58
59      # The dispatcher (CloudRunCompute) supplies the per-execution args:
60      #   infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61      ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

14. assistant (2026-06-21T14:01:02.206Z)

The auth.py file does not exist on this branch (it was in the snapshot git status as untracked `?? backend/api/auth.py`, but I'm on a different branch now). This is relevant to Claim 4 ? let me verify there's no app-layer auth anywhere and check the worker dispatch guard for Claim 2. Let me also locate the cited docs.

15. user (2026-06-21T14:01:04.097Z)

```

<tool_use_error>Path does not exist: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/worker.py. Note: your current working directory is
C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>

```

16. user (2026-06-21T14:01:04.798Z)

No files found

17. user (2026-06-21T14:01:05.272Z)

No files found

18. user (2026-06-21T14:01:05.545Z)

docs\COMPUTE_DECOUPLED_SERVING\README.md

19. assistant (2026-06-21T14:01:08.455Z)

worker.py isn't at backend/worker.py. Let me locate it and the dispatch guard, and read the cited doc lines.

20. user (2026-06-21T14:01:08.955Z)

No files found

21. user (2026-06-21T14:01:09.727Z)

Found 8 files

```
backend\infrastructure\database.py
backend\config\models.py
backend\compute\base.py
backend\worker\__main__.py
backend\compute\cloud_run.py
backend\compute\__init__.py
backend\config\presets.py
backend\config\loader.py
```

22. assistant (2026-06-21T14:01:14.335Z)

worker is a package `backend/worker/\_\_main\_\_.py`. Let me read the dispatch guard there and the cited doc lines (D9, D10, M8).

23. user (2026-06-21T14:01:15.397Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import sys
28     import tempfile
29     from typing import Any, List
30
31     from backend import storage
32     from backend.compute import ComputeJobSpec, get_compute_backend
33     from backend.config import (
34         StartupCheckError,
35         enforce_startup_checks,
36         load_config,
37         probe_cuda_available,
38     )
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.validators.base import registry as validator_registry
42     from backend.reporting import emit_indexer_report
43     from backend.results import Failure
44     from backend.utils.hashing import compute_video_id
45
46     logger = logging.getLogger("backend.worker")
47
48     def _coerce(v: str) -> Any:
```

```

46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
47         low = v.lower()
48         if low in ("true", "false"):
49             return low == "true"
50         for cast in (int, float):
51             try:
52                 return cast(v)
53             except ValueError:
54                 pass
55         return v
56
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")
62             if not sep:
63                 raise ValueError(f"--set expects k=v, got {kv!r}")
64             parts = key.split(".")
65             obj = config
66             for p in parts[:-1]:
67                 obj = getattr(obj, p)
68             setattr(obj, parts[-1], _coerce(val))
69
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily

```

```

imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135             logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                         result.video_id, result.returncode, result.outputs_uri)
137         return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                          status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
144         try:
145             marker = {"video_id": video_id, "returncode": returncode,
146                      "segment_count": segment_count, "status": status}
147             local = os.path.join(scratch, "_done.json")
148             with open(local, "w", encoding="utf-8") as f:
149                 json.dump(marker, f)
150             storage.put(local, done_uri, config=storage_cfg)
151             logger.info("[worker] wrote completion marker -> %s", done_uri)
152         except Exception as e: # never fail a good run because the marker push hiccuped
153             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154

```



```

155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
163         # process.
164         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
165         # through to the
166         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
167         # (the
168         # dispatcher forces compute_target=local_gpu), so it never recursively re-
169         # dispatches.
170         compute = get_compute_backend(config)
171         if compute is not None:
172             return _dispatch_remote(compute, args)
173
174         storage_cfg = config.storage.model_dump()
175
176         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
177         os.makedirs(scratch, exist_ok=True)
178         config.output.output_dir = scratch
179
180         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
181         # to scratch.
182         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
183         config=storage_cfg)
184         print(f"[worker] pulled {args.video_uri} -> {local_video}")
185
186         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
187         # byte-identical).
188         video_id = compute_video_id(local_video)
189
190         # 3. VALIDATE (same registry as the main CLI).
191         val_results, val_context = validator_registry.run_all_with_context(local_video,
192         config)
193         failures = [r for r in val_results if not r.passed]
194         db = Database(os.path.join(scratch, config.output.db_name))
195         db.add_video(video_id, local_video, "failed" if failures else "processed",
196         [r.model_dump() for r in val_results])
197
198         segments: List[dict] = []
199         segmenter_actual = None
200         segmentation_ran = False
201         status = "failed"
202         try:
203             if failures:
204                 print("[worker] validation FAILED: "
205                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
206                 return 2
207             seg_name = args.segmenter or config.indexing.default_segmenter
208             segmenter_cls = segmenter_registry.get(seg_name)
209             if not segmenter_cls:
210                 print(f"[worker] unknown segmenter: {seg_name!r} "
211                 f"(have {list(segmenter_registry.list_available())})")
212                 return 2
213             segmenter_actual = seg_name
214             result = segmenter_cls(db, config).process_video(video_id, local_video,
215             max_frames=args.max_frames)
216             segmentation_ran = True
217             if isinstance(result, Failure):
218                 print(f"[worker] segmenter FAILED: {result.message}")
219                 return 3

```

```

211         segments = result.value if hasattr(result, "value") else result
212         status = "success"
213         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
214         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
215         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
216         if not segments:
217             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
218                             or getattr(config.indexing, "detector_impl", None) or
"auto")
219             logger.warning(
220                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
221                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
222                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
223                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
224                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
225                 video_id, segmenter_actual, detector_impl,
226             )
227         finally:
228             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229             emit_indexer_report(
230                 db=db, video_id=video_id, video_path=local_video, config=config,
231                 val_results=val_results, val_context=val_context,
232                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233                 was_reingest=False, segmentation_ran=segmentation_ran,
234             )
235
236         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
237         out_local = os.path.join(scratch, "outputs", video_id)
238         os.makedirs(out_local, exist_ok=True)
239         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
240         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
241
242         out_prefix = args.out_uri.rstrip("/")
243         pushed = []
244         for fn in sorted(os.listdir(out_local)):
245             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
246             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
247             pushed.append(uri)
248
249         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
250         for u in pushed:
251             print("    ", u)
252
253         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
254         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
255         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
256         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
257         if getattr(args, "done_uri", None):
258             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)

```

```

259         return 0
260
261
262     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
263     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
264     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
265
266
267     def _probe_video_fps_width(path: str):
268         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
269
270         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
271         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
272         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
273         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
274         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
275         rather than guesses.
276         """
277         import json
278         import subprocess
279         try:
280             out = subprocess.check_output(
281                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
282                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
283                 stderr=subprocess.DEVNULL).decode()
284             st = (json.loads(out).get("streams") or [{}])[0]
285             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
286             fps = float(num) / float(den or "1")
287             width = float(st.get("width") or 0)
288             if fps > 0 and width > 0:
289                 return fps, width
290             except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
291                 pass
292             return None
293
294
295     def _resolve_train_detector(args: argparse.Namespace, config: Any):
296         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
297         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
298         stub=CPU mock). Returns the runner, or prints an error + returns None.
299
300         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
301         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
302         no shuttle detector and is rejected.
303         """
304         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
305
306         seg_cls = segmenter_registry.get(args.segmenter)
307         if not seg_cls:
308             print(f"[worker] unknown segmenter: {args.segmenter!r} "
309                   f"(have {list(segmenter_registry.list_available())})")
310             return None
311         seg = seg_cls(None, config)
312         if not isinstance(seg, TrajectoryHybridSegmenter):

```

```

313         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
314               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
315         return None
316     try:
317         runner, _ = seg._resolve_runner(config.get("indexing", {}))
318     except NotImplementedError as e:
319         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
320         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
321         print(f"[worker] detector not available: {e}")
322         return None
323     ok, msg = runner.healthcheck()
324     if not ok:
325         print(f"[worker] detector environment not ready: {msg}")
326         return None
327     print(f"[worker] detector ready ({args.segmenter}): {msg}")
328     return runner
329
330
331 def cmd_train(args: argparse.Namespace) -> int:
332     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
333     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
334
335     Two trajectory sources (the train pipeline + harness are identical either way):
336     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
337         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
338     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
339     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
340         is the M3.5 "first real training" path; only the trajectory source flips.
341     """
342     import subprocess
343
344     config = load_config(args.config) if args.config else load_config()
345     _apply_overrides(config, args.set)
346     if not _run_startup_checks(config):
347         return 2
348     storage_cfg = config.storage.model_dump()
349
350     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
351     labels_dir = os.path.join(scratch, "labels")
352     traj_dir = os.path.join(scratch, "trajectories")
353     videos_dir = os.path.join(scratch, "videos")
354     os.makedirs(labels_dir, exist_ok=True)
355     os.makedirs(traj_dir, exist_ok=True)
356
357     corpus = args.corpus_uri.rstrip("/")
358     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
359                        if u.lower().endswith(".csv"))
360     if len(label_uris) < 2:
361         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
362               f"under {corpus}/labels/")
363         return 2
364
365     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
366     runner = None
367     video_by_stem: dict = {}

```

```

368         if args.trajectory_source == "detector":
369             os.makedirs(videos_dir, exist_ok=True)
370             runner = _resolve_train_detector(args, config)
371             if runner is None:
372                 return 2
373             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
374                 vname = storage.parse_uri(vuri).name
375                 if not vname.lower().endswith(_VIDEO_EXTS):
376                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
377                 video_by_stem[os.path.splitext(vname)[0]] = vuri
378
379             print(f"[worker] trajectory source: {args.trajectory_source}")
380             specs: List[dict] = []
381             for uri in label_uris:
382                 fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
383                 name = fname.replace(".rallies.csv", "").replace(".csv", "")
384                 local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
385                 if args.trajectory_source == "detector":
386                     vuri = video_by_stem.get(name)
387                     if not vuri:
388                         print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
389                         continue
390                     local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
391                                           config=storage_cfg)
392                     video_id = compute_video_id(local_video)
393                     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
394                     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
395                     meta = _probe_video_fps_width(local_video)
396                     if meta is None:
397                         print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
398                         continue
399                     fps, frame_width = meta
400                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
401                     if not traj:
402                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
403                         continue
404                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
405                                "fps": fps, "frame_width": frame_width})
406                 else:
407                     from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
408                     traj = os.path.join(traj_dir, f"{name}_traj.csv")
409                     synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
410                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
411                                "fps": args.fps, "frame_width": args.frame_width})
412
413             if len(specs) < 2:
414                 print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
415                 return 2
416             manifest_path = os.path.join(scratch, "manifest.json")
417             with open(manifest_path, "w", encoding="utf-8") as f:
418                 json.dump(specs, f, indent=2)
419             src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"

```

```

420     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
421
422     out_dir = os.path.join(scratch, "artifacts")
423     cmd = [sys.executable, "-m", "training.gen0.harness",
424            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
425     if args.floor:
426         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
427            config=storage_cfg)
428                        if "://" in args.floor else args.floor)
429         cmd += ["--floor", floor_local]
430     print("[worker] training:", " ".join(cmd))
431     rc = subprocess.run(cmd).returncode
432     if rc != 0:
433         print(f"[worker] gen0 harness failed (rc={rc})")
434         return rc
435
436     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
437     bundle = os.path.join(out_dir, args.version)
438     out_prefix = args.out_uri.rstrip("/")
439     pushed = []
440     for fn in sorted(os.listdir(bundle)):
441         uri = f"{out_prefix}/weights/{args.version}/{fn}"
442         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
443         pushed.append(uri)
444     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
445     for u in pushed:
446         print("    ", u)
447     return 0
448
449 def build_parser() -> argparse.ArgumentParser:
450     p = argparse.ArgumentParser(prog="python -m backend.worker",
451                                description="Storage-aware worker: pull ? run pipeline ?
452     push.")
453     p.add_argument("-v", "--verbose", action="store_true",
454                   help="DEBUG logging (the native detector command, frame counts, per-
455     stage detail)")
456     sub = p.add_subparsers(dest="cmd", required=True)
457
458     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
459     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
460     gcs://")
461     pi.add_argument("--out-uri", required=True, help="output prefix
462     (outputs/<video_id>/? is appended)")
463     pi.add_argument("--segmenter", default=None, help="default:
464     indexing.default_segmenter")
465     pi.add_argument("--config", default=None, help="config.json path (default:
466     ./config.json)")
467     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
468     dir)")
469     pi.add_argument("--max-frames", type=int, default=None)
470     pi.add_argument("--done-uri", default=None,
471                   help="M6: storage URI for a completion MARKER (video_id + rc)
472     written on a "
473
474     "successful run, so a remote (Cloud Run) dispatcher's bucket-
475     poll terminates. "
476
477     "Default-OFF: omit it for the normal local run.")
478     pi.add_argument("--set", action="append", default=[], metavar="k=v",
479                   help="config override, e.g. --set indexing.skip_ai_handoff=true")
480     pi.set_defaults(func=cmd_infer)
481
482     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
483     trajectories -> harness)")
484     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
485     videos/ for "

```

```

473             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
474         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
475         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
476         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
477                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
478                         "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
479                         "(needs videos/ in the corpus; honours detector_impl).")
480         pt.add_argument("--segmenter", default="wasb_hybrid",
481                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
482         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
483         pt.add_argument("--config", default=None)
484         pt.add_argument("--scratch", default=None)
485         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
486         pt.add_argument("--frame-width", type=float, default=1280.0,
487                         help="synth trajectory frame width (detector probes the video)")
488         pt.add_argument("--set", action="append", default=[], metavar="k=v")
489         pt.set_defaults(func=cmd_train)
490         return p
491
492
493     def _setup_logging(verbose: bool) -> None:
494         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
495         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
496         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
497         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
498         entry), not at import, so importing the module never reconfigures a caller's
logging."""
499         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
500         # configured logging at import), which would leave the level at WARNING and re-
suppress the
501         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
502         logging.basicConfig(
503             level=logging.DEBUG if verbose else logging.INFO,
504             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
505             datefmt="%H:%M:%S",
506             force=True,
507         )
508
509
510     def main(argv=None) -> int:
511         args = build_parser().parse_args(argv)
512         _setup_logging(getattr(args, "verbose", False))
513         return args.func(args)
514
515
516     if __name__ == "__main__":
517         sys.exit(main())
518

```

24. assistant (2026-06-21T14:01:19.683Z)

Claim 2's mechanism is confirmed: `cmd\_infer` calls `get\_compute\_backend`, and the dispatched

container forces `local\_gpu` + `native` so it falls through to inline (line 166-168, the guard, plus the `\_DEFAULT\_CONTAINER\_OVERRIDES`). Now let me read the cited doc lines (D9, D10, M8) and the billing.py file.

25. user (2026-06-21T14:01:21.154Z)

30 | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI);
`byo\_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31 | D4 | In **cloud-serving**, STITCH runs on Cloud Run (co-located with detect) so
CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would
hide real cost. |
32 | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory);
detect is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the
rally/window level, never CSV bytes. |
33 | D6 | **Default-OFF**, smallest-safe-first, one PR per milestone; any core-caller
change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-
convention]] | Zero-regression discipline. |
34 | D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an
async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that
~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane
only on a UX complaint. |
35 | D8 | **Pricebook** = a versioned DB table; cost computed in **USD** (matches GCP
billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore
market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |
36 | D9 | **Edge auth** = Cloudflare Access (reuse the site's existing CF). Lock the Cloud
Run **ingress** to the CF proxy + **verify** the `Cf-Access` JWT signature (so a direct hit to
the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity
system. |
37 | D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training
rights** (uploaded footage + derived trajectories/labels); **paid tier** = no-train (privacy is
the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ?
reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on
DERIVED data + **auto-delete raw**; **ToS** counsel-reviewed; live training-on-uploads
gated to M9 (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties
[[paper-coauthor-aim]] / PERSONALIZATION flywheel. |
38 | D11 | **Quality floor:** Gemini handoff ON for cloud-serving until the owned model
clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the
owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a
training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39 | D12 | **Data-flywheel harness** is IN SCOPE (owner add, Q5): capture the (consented,
adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video
workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones**
to the golden TRAINING set (human-reviewed labels) so the owned model climbs toward the D11
floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) +
`data\_pipeline/` + the eval/experiment harness. |
40 | D13 | **Profile** is user-switchable ANYTIME ? a headline FEATURE. The same user runs
local today, cloud tomorrow, back to local ? it's just a config/startup choice; the pure
core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ?
your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the
rally/window level, not bit-identical across GPUs (D5); a single job runs in one profile,
switching is *between* jobs. |
41 | D14 | **NORTHSTAR** ? operate toward this: a **mobile, app-like** flow ? point at a
Google Drive video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel
lands **back in Drive**, next to the source. | owner 2026-06-21 | Implies a **gdrive-api**
(OAuth) StorageBackend (read source + write reel back) ? distinct from the local **mount**
backend; **resolves** \$8 #7, new **M12**. The mobile UI is the khelsutra track; the engine must
support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive
write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC
accommodates it). |
42 | D15 | **Auto-detect SUGGESTS**, the user **CONFIRMS** ? execution is NEVER a surprise. A
GPU owner may still choose cloud; **cloud** (= spend) is never entered silently; the active
profile is always explicit + surfaced. | owner 2026-06-21 | Amends \$4.3 (auto-detect = a
proposed default, not an auto-decision). |
43 | D16 | **M6/M7 implementation SHAPE** + the cheap \$8 picks (owner 2026-06-21): (a)
Cloud Run JOBS (not Services-with-GPU) for the detect+stitch unit, **bucket-poll** for

completion** (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) **stitch ENCODE = libx264** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? **NVDEC for DECODE only** (a speed lever that never touches output bytes); **NVENC encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (\$8 Q8); (c) **input cap = hard-reject`<2GB`** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? \$8 Q9). | owner 2026-06-21 | M6/M7. Resolves \$8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). | 44 | D17 | **Real cloud verification BUDGET = \$100 / ?10k cap (owner 2026-06-21):** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5/M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), **provided the cap is respected** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and **DELETE every resource after** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |

26. user (2026-06-21T14:01:22.062Z)

118 | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory\_hybrid:237,313`, `yolo\_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |
120 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |
[#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-checks+L4 parity ?; runlog ? owner GPU) |
121 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?; real build/run = M7 ? owner) |
122 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
123 | M8 | Per-job **cost ledger** (v6 migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + **pricebook** + aggregation + `api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |
124 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? |
[#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; consent cols + ToS/DPA ? owner/counsel) |
125 | M10 | `byo\_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
126 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data\_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
127 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
128
129 **Testing strategy is a first-class deliverable** ?
[`TESTING\_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[`runlogs/`](runlogs/).

27. assistant (2026-06-21T14:01:26.111Z)

D9 (line 36) and D10 (line 37) confirmed. Important finding: M9 PR #290 exists and the auth IS

implemented (Cf-Access JWT verify + owner_id tag, default-OFF) ? but on the branch I'm on, `backend/api/auth.py` does NOT exist. Let me check the M8 billing.py and also find where the Cf-Access JWT verification lives (it may be on a different branch). Let me check billing.py and search for the auth implementation.

28. user (2026-06-21T14:01:27.114Z)

```
1      """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3      ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4      (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
5      **USD**
6      (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
7      FX rate
8      (the Bangalore market). The pricebook is a versioned DB table
9      (`backend/infrastructure/database.py`);
10     this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
11     dict, so it
12     is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
13
14     **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
15     cost is
16     ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
17     pricebook version.
18     """
19
20     from __future__ import annotations
21
22     from typing import Dict, Mapping, Tuple
23
24     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
25     breakdown.
26     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
27     per gpu_type
28     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
29     billed as GPU)
30     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
31     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
32     not rate-x)
33
34     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
35     USD amount).
36     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
37
38     def compute_cost_usd(usage: Mapping[str, float],
39                          rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
40         """Return ``(cost_usd, breakdown)`` for one job.
41
42         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
43         a resource
44         key to its USD unit price (already resolved for this pricebook version + gpu_type by
45         the DB
46         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
47         multiplied).
48         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
49         is ``0.0``.
50         """
51         breakdown: Dict[str, float] = {}
52         total = 0.0
53         for res in _RATE_MULTIPLIED:
54             amount = float(usage.get(res, 0.0) or 0.0)
55             rate = float(rates.get(res, 0.0) or 0.0)
56             cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
57         the
```

```

43         if amount or rate:                # breakdown reconciles to the total exactly
(no drift),
44         breakdown[res] = cost              # and record the line even at $0 for
transparency
45         total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = `cost_usd × (1 + margin)` . `margin` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

29. user (2026-06-21T14:01:28.505Z)

Found 5 files
tests\test_edge_auth.py
backend\main.py
backend\config\models.py
backend\api\auth.py
artifacts\so.py

30. assistant (2026-06-21T14:01:32.529Z)

So `backend/api/auth.py` DOES exist (the Grep found it). The earlier Read failed ? possibly a working-directory issue. Let me re-read it with the correct absolute path and check the M8 ledger schema for the cost_usd / usage claim. Also verify the billing claim about "egress_gb" and the versioned pricebook.

31. user (2026-06-21T14:01:32.905Z)

```

1     """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant `user_id`
(M9, D9).
2
3     **DEFAULT-OFF:** when `security.edge_auth_enabled` is False the resolver is a strict
NO-OP (returns
4     `None` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5     `Cf-Access-Jwt-Assertion` header is **required** and **verified** (RS256) against the
team JWKS ?
6     signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spooof** the
7     identity header (the explicit D9 anti-spoof requirement). Returns the verified
`user_id` (the
8     token's `email`, else `sub`).
9
10    `jwt` (`PyJWT[crypto]`) is **lazy-imported** so the base runtime works without it
(install the
11    `auth` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12    unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13    """

```

```

14
15     from __future__ import annotations
16
17     import json
18     import time
19     import urllib.request
20     from typing import Any, Callable, Mapping, Optional
21
22     # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23     CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24     _JWKS_PATH = "/cdn-cgi/access/certs"
25     _JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27     JwksFetcher = Callable[[str], dict]
28
29
30     class EdgeAuthError(Exception):
31         """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34     # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35     _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38     def _issuer(team_domain: str) -> str:
39         return team_domain.rstrip("/")
40
41
42     def _default_jwks_fetcher(team_domain: str) -> dict:
43         """Fetch the team's public JWKS (`<team_domain>/cdn-cgi/access/certs`). Network ?
only the
44         production path reaches here; tests pass an injected fetcher."""
45         url = _issuer(team_domain) + _JWKS_PATH
46         with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
47             data: Any = json.load(resp)
48             if not isinstance(data, dict) or "keys" not in data:
49                 raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50             return data
51
52
53     def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54         """Return the JWKS for `team_domain`, cached with a TTL. An injected `fetcher`
(tests)
55         bypasses the cache so each test is deterministic."""
56         if fetcher is not None:
57             return fetcher(team_domain)
58         now = time.time()
59         cached = _jwks_cache.get(team_domain)
60         if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61             return cached[1]
62         jwks = _default_jwks_fetcher(team_domain)
63         _jwks_cache[team_domain] = (now, jwks)
64         return jwks
65
66
67     def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
68         """Verify a Cf-Access JWT against `jwks` (RS256; checks sig + exp + issuer +
audience) and
69         return the user_id (`email` else `sub`). Raises `EdgeAuthError` on ANY

```

```

failure."""
70         try:
71             import jwt
72             from jwt import PyJWKSet
73         except Exception as exc: # pragma: no cover - only when the auth extra isn't
installed
74             raise EdgeAuthError(
75                 "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76                 "(pip install -e .[auth]) on a host with edge_auth_enabled."
77             ) from exc
78
79         try:
80             kid = jwt.get_unverified_header(token).get("kid")
81             if not kid:
82                 raise EdgeAuthError("token header has no kid")
83             signing_key = PyJWKSet.from_dict(jwks)[kid]
84             claims = jwt.decode(
85                 token, signing_key.key, algorithms=["RS256"],
86                 audience=aud, issuer=_issuer(team_domain),
87                 options={"require": ["exp", "iss", "aud"]},
88             )
89         except EdgeAuthError:
90             raise
91         except KeyError as exc: # kid not in the JWKS
92             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93         except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94             raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96         user = claims.get("email") or claims.get("sub")
97         if not user:
98             raise EdgeAuthError("verified token carries no email/sub claim")
99         return str(user)
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                       jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
107         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.
108         ``jwks_fetcher`` is injectable for offline tests.
109         """
110         sec = getattr(config, "security", None)
111         if not sec or not getattr(sec, "edge_auth_enabled", False):
112             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
113
114         team_domain = getattr(sec, "cf_team_domain", "") or ""
115         aud = getattr(sec, "cf_access_aud", "") or ""
116         if not team_domain or not aud:
117             raise EdgeAuthError(
118                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121         # dict too (tests) by trying both casings.
122         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
123         if not token:
124             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
125
126         jwks = _get_jwks(team_domain, jwks_fetcher)

```

```
127         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
128
```

32. assistant (2026-06-21T14:01:37.392Z)

This confirms the M9 auth implementation is on this branch. Note: the JWKS TTL here is 600s (10 min) with the comment "keys rotate slowly" ? relevant to Claim 6's "rotates ~every 6 weeks". The implementation refreshes every 10 min regardless. Now let me check the M8 ledger DB schema for the cost columns and pricebook, and the create_gpu_vm.sh / teardown.sh for Claim 10.

33. user (2026-06-21T14:01:39.412Z)

```
backend\infrastructure\database.py:295:         # billing.enabled + a real pricebook) + a
VERSIONED `pricebook` table. Cost is stored in
backend\infrastructure\database.py:303:         ("gpu_active_seconds", "REAL"),
backend\infrastructure\database.py:304:         ("wall_seconds", "REAL"),
backend\infrastructure\database.py:305:         ("bytes_out", "INTEGER"),             # egress
backend\infrastructure\database.py:306:         ("gemini_cost_usd", "REAL"),         #
provider-reported, already USD
backend\infrastructure\database.py:307:         ("pricebook_version", "TEXT"),
backend\infrastructure\database.py:308:         ("cost_usd", "REAL"),                 #
computed total (USD = source of truth)
backend\infrastructure\database.py:309:         ("cost_breakdown_json", "TEXT"),     #
{gpu_seconds: .., egress_gb: .., gemini_usd: ..}
backend\infrastructure\database.py:315:         # Versioned pricebook: one row per (version,
resource, gpu_type). Rates change + differ by
backend\infrastructure\database.py:318:         CREATE TABLE IF NOT EXISTS pricebook (
backend\infrastructure\database.py:327:         cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS
idx_pricebook_key "
backend\infrastructure\database.py:328:                        "ON pricebook(version, resource,
gpu_type)")
backend\infrastructure\database.py:329:         # Seed a PLACEHOLDER version with every rate 0.0
? cost_usd computes to 0 (nothing
backend\infrastructure\database.py:331:         # egress $/GB) and flips
billing.pricebook_version to it ? never overwrite this seed.
backend\infrastructure\database.py:333:                        ("wall_seconds", ""),
("egress_gb", "")):
backend\infrastructure\database.py:335:         "INSERT OR IGNORE INTO pricebook
(version, resource, gpu_type, unit_price_usd, "
backend\infrastructure\database.py:336:         "currency) VALUES ('placeholder-v0', ?,
?, 0.0, 'USD')", (resource, gpu_type))
backend\infrastructure\database.py:605:     def get_pricebook_rates(self, version: str, gpu_type:
Optional[str] = None) -> Dict[str, float]:
backend\infrastructure\database.py:606:         """Resolve a flat ``{resource: unit_price_usd}``
for a pricebook ``version``. The per-second
backend\infrastructure\database.py:613:         "SELECT resource, gpu_type,
unit_price_usd FROM pricebook WHERE version = ?",
backend\infrastructure\database.py:624:     def record_job_cost(self, job_id: str, *, usage:
Dict[str, float], pricebook_version: str,
backend\infrastructure\database.py:627:         """Compute the job's cost from ``usage`` x the
pricebook + persist every cost column.
backend\infrastructure\database.py:630:         rates =
self.get_pricebook_rates(pricebook_version, gpu_type)
backend\infrastructure\database.py:638:         "[BILLING] no pricebook rate for
gpu_type=%r in version=%r ? %.1f GPU-seconds billed "
backend\infrastructure\database.py:639:         "at $0 (UNPRICED; recorded in
cost_breakdown_json). Add a rate or fix gpu_type.",
backend\infrastructure\database.py:640:         gpu_type, pricebook_version, gpu_amt)
backend\infrastructure\database.py:641:         cost_usd, breakdown =
billing.compute_cost_usd(usage, rates)
backend\infrastructure\database.py:645:         marked = billing.apply_margin(cost_usd,
margin)
backend\infrastructure\database.py:646:         breakdown["margin"] = round(marked -
cost_usd, 6) # keep ?(breakdown) == cost_usd
backend\infrastructure\database.py:647:         cost_usd = marked
```

```

backend\infrastructure\database.py:648:         bytes_out =
int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
backend\infrastructure\database.py:650:         "UPDATE jobs SET owner_id=?,
compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
backend\infrastructure\database.py:651:         "wall_seconds=?, bytes_out=?,
gemini_cost_usd=?, pricebook_version=?, cost_usd=?, "
backend\infrastructure\database.py:652:         "cost_breakdown_json=? WHERE id = ?",
backend\infrastructure\database.py:654:         usage.get(billing.WALL_SECONDS), bytes_out,
usage.get(billing.GEMINI_USD),
backend\infrastructure\database.py:655:         pricebook_version, cost_usd,
json.dumps(breakdown), job_id),
backend\infrastructure\database.py:659:         return {"cost_usd": cost_usd, "cost_breakdown":
breakdown,
backend\infrastructure\database.py:660:         "pricebook_version": pricebook_version}
backend\infrastructure\database.py:663:         """The job's cost row (`None` if the job
doesn't exist; `cost_usd` is `None` until
backend\infrastructure\database.py:667:         "SELECT id, video_id, owner_id,
compute_backend, gpu_type, gpu_active_seconds, "

```

[Showing results with pagination = limit: 40]

34. user (2026-06-21T14:01:40.003Z)

```

1      #!/usr/bin/env bash
2      # Create a rally GPU VM that ALWAYS installs the Ops Agent (Observability). This is THE
one way to
3      # spin up a rally GPU box ? it bakes the ops-agent startup-script into the VM metadata,
attaches the
4      # rally-worker service account (+ cloud-platform scope so it can read/write the bucket +
report
5      # metrics), and uses a driver-ready Deep-Learning-VM image (nvidia-smi works
immediately).
6      #
7      # It sweeps zones for GPU capacity ? asia-south1 (Mumbai, co-located with the bucket)
first, then
8      # asia-southeast1 (Singapore), which L4 frequently falls back to when Mumbai is stocked-
out.
9      #
10     #   bash deploy/gcp/create_gpu_vm.sh                               # L4 (default), name
rally-gpu
11     #   RALLY_VM_NAME=rally-m7 RALLY_GPU=t4 bash deploy/gcp/create_gpu_vm.sh
12     #
13     # Prereq (once per project): bash deploy/gcp/provision.sh   (enables logging/monitoring
+ grants the
14     # SA the Ops-Agent writer roles) AND the GPUS_ALL_REGIONS quota >= 1 (README S2).
15     #
16     # ? DELETE when done (a running GPU VM is the only real cost):
17     #   RALLY_GCP_ZONE=<zone-it-printed> bash deploy/gcp/teardown.sh delete
18     set -uo pipefail
19
20     PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
21     NAME="${RALLY_VM_NAME:-rally-gpu}"
22     SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
23     GPU="${RALLY_GPU:-l4}"      # l4 | t4
24     HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
25     STARTUP="$HERE/ops-agent-startup.sh"
26     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
27
28     ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
29
30     if [ "$GPU" = "l4" ]; then
31         MACHINE="g2-standard-4"; ACCEL=( )           # L4 is integral to the g2 machine family
32     else
33         MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1)

```

```

34     fi
35
36     ERR="$(mktemp)"
37     for Z in $ZONES; do
38         echo "== trying $NAME ($GPU) in $Z =="
39         if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
40             --machine-type="$MACHINE" "${ACCEL[@]}" \
41             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
42             --image-family=common-cu129-ubuntu-2204-nvidia-580 \
43             --image-project=deeplearning-platform-release \
44             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
45             --service-account="$SA" --scopes=cloud-platform \
46             --metadata-from-file=startup-script="$STARTUP" 2>"$ERR"; then
47             echo "? CREATED $NAME in $Z ? the Ops Agent installs on boot (Observability ~2-3 min
later)."
48             echo "    SSH:    gcloud compute ssh $NAME --zone=$Z"
49             echo "    DELETE: RALLY_GCP_ZONE=$Z bash deploy/gcp/teardown.sh delete"
50             rm -f "$ERR"; exit 0
51         fi
52         if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then
53             echo "    (no $GPU capacity in $Z ? trying the next zone)"; continue
54         fi
55         echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
56     done
57     echo "? No $GPU capacity in any swept zone ($ZONES). Try again later or RALLY_GPU=t4."
58     rm -f "$ERR"; exit 1
59

```

35. assistant (2026-06-21T14:01:46.525Z)

Claim 9 (create_gpu_vm.sh) confirmed: L4 default, zone sweep asia-south1 ? asia-southeast1, bakes ops-agent + rally-worker SA. Note: the column is `gpu\_active\_seconds` in the DB but `gpu\_seconds` in billing.py usage dict ? Claim 10 uses "gpu_seconds" which matches the billing usage key. Let me verify teardown.sh and check the LOCAL_APPLIANCE_ARCHITECTURE doc for the per-video cap "open owner question" (recommendation point 2), and the \$10 reference.

36. user (2026-06-21T14:01:46.944Z)

```

1     #!/usr/bin/env bash
2     # Switch OFF the billable GCP resources between sessions (cost control). The GPU VM is
the only
3     # meaningful cost. ? A STOPPED VM STILL BILLS FOR ITS DISK (pd-ssd ? $0.17/GB/mo ? a 200
GB boot
4     # disk ? $34/mo) ? only DELETE takes it to $0. So DELETE is the DEFAULT here (owner cost
rule,
5     # 2026-06-20: "make sure VMs are deleting, not just stopped"); use `stop` only for a
deliberate
6     # fast-resume pause. The GCS bucket is NEVER auto-deleted (it holds the corpus) ? drop
it manually.
7     #
8     # Usage:
9     #   bash deploy/gcp/teardown.sh                # DELETE the VM (default; $0 ? removes VM +
boot disk)
10    #   bash deploy/gcp/teardown.sh delete          # delete the VM (no disk cost; full re-port
to resume)
11    #   bash deploy/gcp/teardown.sh stop            # STOP only ? keeps the disk ? STILL BILLS
(~$mo!); fast resume
12    #   bash deploy/gcp/teardown.sh all             # delete VM + SA + budget (keeps the
bucket/data)
13    #
14    # Cross-region VM: set the zone, e.g. RALLY_GCP_ZONE=asia-southeast1-a bash
deploy/gcp/teardown.sh
15    set -uo pipefail
16
17    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"

```



```

18     ZONE="${RALLY_GCP_ZONE:-asia-south1-a}"
19     VM="${RALLY_GPU_VM:-rally-gpu}"
20     SA_EMAIL="rally-worker@${PROJECT}.iam.gserviceaccount.com"
21     BILLING_ACCT="${RALLY_BILLING_ACCT:-01420D-419EE0-53D83C}"
22     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
23     ACTION="${1:-delete}"
24
25     case "$ACTION" in
26         stop)
27             echo "== stopping $VM ? ? the DISK STILL BILLS (~\$/mo); use 'delete' for \$0.
Resume: gcloud compute instances start $VM --zone=$ZONE =="
28             gcloud compute instances stop "$VM" --zone="$ZONE" --project "$PROJECT" || echo "
(no running VM named $VM)"
29             ;;
30         delete)
31             echo "== deleting $VM (VM + boot disk autoDelete=true => \$0; full re-port to
resume) =="
32             gcloud compute instances delete "$VM" --zone="$ZONE" --project "$PROJECT" --quiet ||
echo " (no VM named $VM)"
33             echo " audit orphans: gcloud compute disks list --project $PROJECT (expect NO
unattached disk ? it would still bill)"
34             ;;
35         all)
36             echo "== deleting VM + SA + budget (bucket/data kept) =="
37             gcloud compute instances delete "$VM" --zone="$ZONE" --project "$PROJECT" --quiet ||
echo " (no VM)"
38             gcloud iam service-accounts delete "$SA_EMAIL" --project "$PROJECT" --quiet || echo
" (no SA)"
39             bid=$(gcloud billing budgets list --billing-account="$BILLING_ACCT" \
40                 --filter="displayName=rally-100usd-guard" --format="value(name)" 2>/dev/null |
head -1)
41             [ -n "$bid" ] && gcloud billing budgets delete "$bid" --billing-
account="$BILLING_ACCT" --quiet || echo " (no budget)"
42             echo " bucket gs://khesutra-rally-corpus left intact (delete manually: gcloud
storage rm -r gs://khesutra-rally-corpus)"
43             ;;
44         *)
45             echo "usage: $0 {stop|delete|all}"; exit 2 ;;
46     esac
47     echo "done."
48

```

37. user (2026-06-21T14:01:47.644Z)

No files found

38. assistant (2026-06-21T14:01:52.848Z)

teardown.sh confirms: delete is the default, stopped VM still bills disk. Note the comment says "\$34/mo" for 200GB pd-ssd, but the create script uses 120GB pd-balanced ? a minor inconsistency, but the claim's "\$34/mo" matches teardown.sh's own comment. The claim is about the doc's stated rule, which is accurate.

LOCAL_APPLIANCE_ARCHITECTURE doc wasn't found by glob. Let me search for it and the per-video cap reference. This relates to the recommendation, not a numbered claim, but worth checking.

39. user (2026-06-21T14:01:53.917Z)

Found 1 file
docs\COMPUTE_DECOUPLED_SERVING\README.md

40. user (2026-06-21T14:01:54.757Z)

290 cursor.execute("PRAGMA user_version = 5")

```

291
292     def _migrate_to_v6(self, cursor):
293         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNS are
298         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299         for col, typ in (
300             ("owner_id", "TEXT"),          # who pays (the non-owner user / tenant)
301             ("compute_backend", "TEXT"),    # local_gpu | cloud_run | ...
302             ("gpu_type", "TEXT"),          # L4 | T4 | ... (selects the per-second
GPU rate)
303             ("gpu_active_seconds", "REAL"),
304             ("wall_seconds", "REAL"),
305             ("bytes_out", "INTEGER"),       # egress
306             ("gemini_cost_usd", "REAL"),    # provider-reported, already USD
307             ("pricebook_version", "TEXT"),
308             ("cost_usd", "REAL"),          # computed total (USD = source of truth)
309             ("cost_breakdown_json", "TEXT"), # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
310         ):
311             self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312         # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314                        "WHERE owner_id IS NOT NULL")
315         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317         cursor.execute("""
318             CREATE TABLE IF NOT EXISTS pricebook (
319                 version TEXT NOT NULL,
320                 resource TEXT NOT NULL,
321                 gpu_type TEXT NOT NULL DEFAULT '',
322                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
323                 currency TEXT NOT NULL DEFAULT 'USD',
324                 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325             )
326         """)
327         cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                        "ON pricebook(version, resource, gpu_type)")
329         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing

```

41. assistant (2026-06-21T14:01:59.898Z)

The M8 ledger claim is fully confirmed: usage (gpu_seconds, wall_seconds, egress_gb) × rate from versioned pricebook + gemini_usd passthrough, USD internal/INR display, default-OFF with placeholder-v0 (all 0.0). The "worker usage-emission is a follow-up" is confirmed by the M8 row note ("usage-emission ? owner/follow-up").

Now I have verified all the in-repo code claims. Let me turn to the external/web claims (5, 6, 7, 8, 13, 14) and the WASB licensing claim (12). Let me load WebSearch/WebFetch and start the searches.

42. assistant (2026-06-21T14:02:03.823Z)

Now let me run the external fact-checking searches. I'll batch the independent ones.

43. user (2026-06-21T14:02:11.707Z)

Web search results for query: "Cloud Run NVIDIA L4 GPU pricing per second 2026 scale to zero minimum 4 CPU 16Gi"

Links: [{"title": "Cloud Run pricing | Google Cloud", "url": "https://cloud.google.com/run/pricing"}, {"title": "L4 Cloud Pricing: Compare 15+ Providers (2026)", "url": "https://getdeploying.com/gpus/nvidia-l4"}, {"title": "GPU support for services | Cloud Run | Google Cloud Documentation", "url": "https://docs.cloud.google.com/run/docs/configuring/services/gpu"}, {"title": "Google Cloud Run Pricing in 2025: A Comprehensive Guide", "url": "https://cloudchipr.com/blog/cloud-run-pricing"}, {"title": "VM instance pricing | Google Cloud", "url": "https://cloud.google.com/compute/gpus-pricing"}, {"title": "NVIDIA L4 GPU: Price, Specs & Cloud Pricing Guide (2026)", "url": "https://jarvislabs.ai/blog/l4-gpu-price"}, {"title": "Top Serverless GPU Clouds for 2026: Comparing Runpod, Modal, and More", "url": "https://www.runpod.io/articles/guides/top-serverless-gpu-clouds"}, {"title": "L4 GPU Cloud | Runpod", "url": "https://www.runpod.io/gpu-models/l4"}, {"title": "GPU Cloud Pricing | Per-Second H100, A100, RTX | Runpod", "url": "https://www.runpod.io/pricing"}, {"title": "How much does it cost to run NVIDIA L4 GPUs in 2025?", "url": "https://modal.com/blog/nvidia-l4-price-article"}]

Based on the search results, I found relevant information about Cloud Run NVIDIA L4 GPU pricing with the specific configuration you mentioned:

Cloud Run NVIDIA L4 GPU Pricing (2026)

For Tier 1 regions, an NVIDIA L4 costs around \$0.0001867 per second without zonal redundancy, which is roughly \$0.67 per hour. With zonal redundancy, pricing is higher (about \$0.0002909 per second).

Configuration Requirements

For L4 GPUs, you must use a minimum of 4 CPU and 16 GiB of memory ? which matches exactly the specifications in your query.

Scale to Zero Capability

Instances of a Cloud Run service that has been configured to use GPU can scale down to zero for cost savings when not in use. Autoscaling can still reduce instances to zero when traffic stops, helping control cost between bursts.

Billing Model

GPU usage is billed per second separately from CPU and memory, and there is no free tier for GPUs. GPU-enabled Cloud Run services require CPU to remain allocated during instance lifetime, and you pay for the full lifetime per second while the instance is running.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

44. user (2026-06-21T14:02:13.304Z)

Web search results for query: "Cloudflare Access JWT validation PyJWT JWKS cdn-cgi/access/certs key rotation 6 weeks RS256 aud"

Links: [{"title": "Validate JWTs · Cloudflare One docs", "url": "https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/validating-json/"}, {"title": "Validating JSON Web Tokens · Cloudflare Access docs", "url": "https://developers.cloudflare.com/access/setting-up-access/validate-jwt-tokens"}, {"title": "JSON Web Tokens validation · Cloudflare API Shield docs", "url": "https://developers.cloudflare.com/api-shield/security/jwt-validation/"}, {"title": "Configure the Worker for JWT validation · Cloudflare API Shield"}]

```
docs", "url": "https://developers.cloudflare.com/api-shield/security/jwt-validation/jwt-
worker/"), {"title": "Cloudflare Zero Trust Site
Protection", "url": "https://www.aem.live/developer/cloudflare-zero-trust"}, {"title": "GitHub -
ymyzk/cla-jwt-verifier: Simple HTTP server for verifying JWT issued by Cloudflare
Access", "url": "https://github.com/ymyzk/cla-jwt-verifier"}, {"title": "Cloudflare Zero Trust and
PHP. If you are, just like I was, looking? | by Henze Berkheij |
Medium", "url": "https://medium.com/@henzeberkheij/cloudflare-zero-trust-and-
php-40b69e9d2c86"}, {"title": "Validate the Access token with FastAPI · Cloudflare One
docs", "url": "https://developers.cloudflare.com/cloudflare-
one/tutorials/fastapi/"}, {"title": "Using Cloudflare Access and devise together. - Ellery
Durgin", "url": "https://www.durgin.dev/using-cloudflare-teams-and-devise-
together/"}, {"title": "One-click Cloudflare Access for Workers ·
Changelog", "url": "https://developers.cloudflare.com/changelog/post/2025-10-03-one-click-access-
for-workers/"}]]
```

Based on the search results, here's comprehensive information about Cloudflare Access JWT validation:

JWT Validation Overview

Cloudflare signs a JSON Web Token (JWT) when users or services authenticate through Cloudflare Access. Cloudflare signs both tokens with a key pair that you can validate using the corresponding public certificate, available at an external endpoint.

JWKS Endpoint and Key Management

The endpoint `https://<your-team-name>.cloudflareaccess.com/cdn-cgi/access/certs` contains two public keys: the current key used to sign all new tokens, and the previous key that has been rotated out.

Key Rotation

Cloudflare rotates the key pair in use periodically. For this reason, please only validate tokens using the external endpoint, rather than saving the public key as a hard-coded value. Match the kid value in the JWT to the corresponding certificate in `public_certs`.

PyJWT Implementation

PyJWT can use a list of RSA public keys. The `CERTS_URL` format is `"{/cdn-cgi/access/certs".format(AUTH_DOMAIN)`

JWT Claims Validation

When validating JWTs, you should verify:

- The `iss` field should point to your team domain (`https://<your-team-name>.cloudflareaccess.com`)
- The ``aud`` (audience) claim against your policy's audience tag
- The RS256 algorithm

The search results confirm the core components you asked about: PyJWT, JWKS validation, the `cdn-cgi/access/certs` endpoint, and RS256 signature verification. However, the specific "6 weeks" key rotation interval was not mentioned in these results.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

45. assistant (2026-06-21T14:02:19.876Z)

Important nuances emerging:

- **Claim 7** (Cloud Run L4): The pricing ``$0.0001867/sec`` (~\$0.67/hr), per-second, scale-to-zero, 24 CPU + 16Gi all confirmed. BUT the claim says "GPU-ready in ~5s with drivers pre-

installed." Need to verify that figure. Also the claim cites Cloud Run **jobs** GPU docs while the code uses Jobs (D16). The search returned services GPU docs. Let me verify the ~5s startup figure and the jobs GPU support.

- **Claim 6** (Cloudflare): "rotates ~every 6 weeks" was NOT confirmed by search ? Cloudflare docs say "periodically" without a specific interval. This makes the "6 weeks" specific claim **uncertain**. Also the claim says "aud + RS256" which is confirmed.

Let me verify the ~5s GPU startup and Cloud Run Jobs GPU, and the Render/Railway/Fly GPU claims.

46. user (2026-06-21T14:02:25.864Z)

Web search results for query: "Fly.io GPU deprecated discontinued August 2025 Render Railway no GPU support"

Links: [{"title": "Fly GPUs · Fly Docs", "url": "https://fly.io/docs/gpus/"}, {"title": "Fly.io Resource Pricing · Fly Docs", "url": "https://fly.io/docs/about/pricing/"}, {"title": "GPU migration - \"Fly.io GPUs will be deprecated as of July 31, 2026\" - login-only - Fly.io", "url": "https://community.fly.io/t/gpu-migration-fly-io-gpus-will-be-deprecated-as-of-july-31-2026/27110"}, {"title": "Using GPUs with FKS · Fly Docs", "url": "https://fly.io/docs/kubernetes/using-gpus/"}, {"title": "Getting Started with Fly GPUs · Fly Docs", "url": "https://fly.io/docs/gpus/getting-started-gpus/"}, {"title": "Python GPU Dev Machine · Fly Docs", "url": "https://fly.io/docs/gpus/python-gpu-example/"}, {"title": "Fly GPUs quickstart · Fly Docs", "url": "https://fly.io/docs/gpus/gpu-quickstart/"}, {"title": "Scaling Large Language Models to zero with Ollama · The Fly Blog", "url": "https://fly.io/blog/scaling-llm-ollama/"}, {"title": "We Were Wrong About GPUs · The Fly Blog", "url": "https://fly.io/blog/wrong-about-gpu/"}, {"title": "Fly.io has GPUs now · The Fly Blog", "url": "https://fly.io/blog/fly-io-has-gpus-now/"}]

Based on the search results, here's what I found regarding Fly.io GPUs:

Fly.io GPU Deprecation

GPUs are deprecated and will be unavailable after August 1. However, there's an important clarification: Fly.io GPUs will be fully deprecated as of July 31, 2026 (not August 2025 as mentioned in your query).

The deprecation was explained in a blog post noting that developers don't want GPUs, and Fly.io determined that GPU workloads were not core to their platform's mission.

Render and Railway GPU Support

Your query also asked about Render and Railway's GPU support. The search results provided don't contain information about Render or Railway's GPU offerings or policies. To get accurate information about their current GPU support status, you would need to check their official documentation or website directly.

Would you like me to search for information about Render and Railway's GPU offerings?

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

47. user (2026-06-21T14:02:26.051Z)

Web search results for query: "Cloud Run GPU jobs L4 instance startup time 5 seconds drivers preinstalled cold start"

Links: [{"title": "Configure GPUs for Cloud Run jobs | Google Cloud Documentation", "url": "https://docs.cloud.google.com/run/docs/configuring/jobs/gpu"}, {"title": "GPU support for services | Cloud Run | Google Cloud Documentation", "url": "https://docs.cloud.google.com/run/docs/configuring/services/gpu"}, {"title": "Cloud Run GPUs are now generally available | Google Cloud Blog", "url": "https://cloud.google.com/blog/products/serverless/cloud-run-gpus-are-now-generally-available"}, {"title": "Host your LLMs on Cloud Run | Google Cloud"}]

```
Blog", "url": "https://cloud.google.com/blog/products/application-development/run-your-ai-inference-applications-on-cloud-run-with-nvidia-gpus?hl=en"}, {"title": "GPU support for worker pools | Cloud Run | Google Cloud Documentation", "url": "https://docs.cloud.google.com/run/docs/configuring/workerpools/gpu"}, {"title": "A Guide to AI Cold Starts on Cloud Run | Google Cloud Blog", "url": "https://cloud.google.com/blog/topics/developers-practitioners/a-guide-to-ai-cold-starts-on-cloud-run"}, {"title": "Google Cloud Run Now Offers Serverless GPUs for AI and Batch Processing - InfoQ", "url": "https://www.infoq.com/news/2025/06/google-cloud-run-nvidia-gpu/"}, {"title": "Supercharging Cloud Run with GPU Power: A New Era for AI Workloads | by Rahul Kumar Singh | Google Cloud - Community | Medium", "url": "https://medium.com/google-cloud/supercharging-cloud-run-with-gpu-power-a-new-era-for-ai-workloads-3c54fcf60cae"}]
```

Based on the search results, here's what I found about Cloud Run GPU jobs with L4 instances and startup time:

Startup Time

Cloud Run instances with an attached L4 GPU with drivers pre-installed start in approximately 5 seconds, at which point the processes running in your container can start to use the GPU. This applies to both Cloud Run jobs and services.

Pre-installed Drivers

GPU on Cloud Run is fully managed, with no extra drivers or libraries needed. The drivers come pre-installed as part of the managed service offering.

Cold Start Performance

For AI inference workloads specifically, while the GPU instance itself starts in ~5 seconds, the total cold start time is longer when you include model loading. After the GPU starts, you'll need another few seconds for the framework and model to load and initialize. Cold-start times for Gemma 2b, Gemma2 9b, Llama2 7b/13b, and Llama3.1 8b models with the Ollama framework range from 11 to 35 seconds.

Configuration Requirements

For L4 GPU instances, you must specify at least 4 CPU and at least 16Gi (16 GiB) of memory.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

48. assistant (2026-06-21T14:02:32.755Z)

Key findings:

- **Claim 7** fully confirmed including "~5s GPU-ready with drivers pre-installed" ? confirmed for both jobs and services. The docs.cloud.google.com/run/docs/configuring/jobs/gpu URL is valid.
- **Claim 8** (Fly GPU deprecated "after Aug 1"): The deprecation date is **July 31, 2026** (per Fly community post), meaning unavailable after Aug 1, **2026** ? NOT Aug 1 2025. The claim says "after Aug 1" without a year, and the recommendation says "after Aug 1." Given today is June 2026, "after Aug 1" reads as Aug 1 2026, which matches. So the date is essentially correct (Aug 1 2026), though the source URL in the claim (fly.io/docs/gpus/) is right but the specific date detail. The Render/Railway "no GPU" part still needs verification. Let me confirm Render and Railway GPU status and the Deploy-to-Render blueprint claim.

49. user (2026-06-21T14:02:39.818Z)

Web search results for query: "Railway GPU support 2026 does Railway offer GPU"

Links: [{"title": "GPU Support - Railway Central Station", "url": "https://station.railway.com/feedback/gpu-support-56d19c42"}, {"title": "GPU

Acceleration For Autonomous Trains", "url": "https://www.meegle.com/en_us/topics/gpu-acceleration/gpu-acceleration-for-autonomous-trains"}, {"title": "Railway vs Render: which platform fits your workload in 2026? | Blog ? Northflank", "url": "https://northflank.com/blog/railway-vs-render"}, {"title": "Railway", "url": "https://station.railway.com/feedback/plan-for-gpu-support-b9fddca5"}, {"title": "Upstart cloud provider Railway turns heads with speed | TechTarget", "url": "https://www.techtarget.com/searchcloudcomputing/news/366637659/Upstart-cloud-provider-Railway-turns-heads-with-speed"}, {"title": "Railway | Review, Pricing & Alternatives", "url": "https://getdeploying.com/railway"}, {"title": "GPUaaS vs Railway", "url": "https://getdeploying.com/gpuaaS-vs-railway"}, {"title": "Railway Grade Computers - Forecr", "url": "https://www.forecr.io/collections/railway-grade-computers"}, {"title": "Plans for GPUs? - Railway Central Station", "url": "https://station.railway.com/questions/plans-for-gpus-3cbb0f62"}, {"title": "Railway Computer | Rugged Fanless PC | EN50155/EN45545 Certified - Neosys Technology", "url": "https://www.neosys-tech.com/en/product/product-lines/railway-computer"}]

Based on the search results, here's what I found about Railway's GPU support:

Current Status: No GPU Support

Railway does not provide GPU workload support. This is confirmed by multiple sources indicating that as of 2026, Railway's infrastructure focuses on CPU-optimized deployments.

CEO's Position on Future GPU Support

Railway's CEO stated they "have the full intention of becoming the best intelligent cloud for integrating with and running AI applications, but cannot disclose explicit GPU plans at this time." This suggests GPU support is a consideration for the future, though no timeline has been announced.

Historical Context

An unofficial comment from someone familiar with Railway's infrastructure suggested GPUs were "on order for our move to metal," though there was still no ETA provided. This comment was made earlier and appears to have not materialized into announced GPU offerings yet.

Impact

Industry analysts noted that the lack of GPU support could hinder Railway's efforts to capture more AI workloads, which may be driving internal discussions about eventually adding this capability.

If you need GPU support for your workloads, you would currently need to use alternative platforms that offer dedicated GPU services.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

50. user (2026-06-21T14:02:44.868Z)

Web search results for query: "Render.com GPU support 2026 does Render offer GPU instances"

Links: [{"title": "GPU instances | Feature Requests | Render", "url": "https://feedback.render.com/features/p/gpu-instances"}, {"title": "Does render offer GPUs? - Render", "url": "https://render.discourse.group/t/does-render-offer-gpus/11222"}, {"title": "Latest Render News - (RENDER) Future Outlook, Trends & Market Insights", "url": "https://coinmarketcap.com/cmc-ai/render/latest-updates/"}, {"title": "Render Network Review 2026: Decentralized GPU Rendering For Creators And AI Workloads", "url": "https://cryptoadventure.com/render-network-review-2026-decentralized-gpu-rendering-for-creators-and-ai-workloads/"}, {"title": "Can RENDER Ride the AI Wave in 2026? | Disruption Banking", "url": "https://www.disruptionbanking.com/2026/02/19/can-render-ride-the-ai-wave-in-2026/"}, {"title": "Render"}]

```
Network", "url": "https://rendernetwork.com/"), {"title": "Understanding the Render Network: A Comprehensive Overview | Messari", "url": "https://messari.io/report/understanding-the-render-network-a-comprehensive-overview"}, {"title": "Do render.com offer any hosting solution for services that require VRAM? - Render", "url": "https://community.render.com/t/do-render-com-offer-any-hosting-solution-for-services-that-require-vram/6425"}, {"title": "Render Network Foundation Monthly Report ? March 2026 | by Render Network | Medium", "url": "https://rendernetwork.medium.com/render-network-foundation-monthly-report-march-2026-f598560bdf30"}]
```

Based on the search results, I need to clarify an important distinction: **Render.com** (a web hosting platform) and **Render Network** (a decentralized GPU computing platform) are two different services.

Regarding Render.com: The search results show community discussions asking whether render.com offers GPU hosting, but there's no evidence that render.com itself currently offers GPU instances as part of their hosting services.

Regarding Render Network (a different service): The Render Network is the world's first decentralized GPU rendering platform, and it does offer GPU support. As of October 2025, Render Network proposed adding support for enterprise-grade GPUs like NVIDIA H100/H200 and AMD MI300 series. The Render Network combines support for the leading GPU render engines OctaneRender, Redshift, and Blender Cycles with the latest generative AI imaging tools.

If you were asking about Render.com's hosting services specifically, based on the available information, they do not appear to offer dedicated GPU instances. If you meant to ask about the Render Network (the decentralized GPU platform), it does provide GPU compute resources and is actively expanding its GPU support in 2026.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

51. assistant (2026-06-21T14:02:49.438Z)

Confirmed: Render.com (the PaaS) does not offer GPU instances; Railway does not provide GPU support as of 2026. Claim 8 holds. Now let me verify the "Deploy to Render" Blueprint claim (Claim 9 ? render.yaml provisions on the user's account on click, autoDeploy:false advice) and the WASB license claim, plus the DPDP Act facts.

52. user (2026-06-21T14:02:53.547Z)

Deploy to Render Button Overview

Yes, the Deploy to Render button works with a render.yaml Blueprint. When users click it, they "review the services to be deployed on their Render account and click approve to set everything up instantly."

Regarding autoDeploy settings, the documentation states: "For a service that is meant to be deployed via a Deploy to Render button, it is strongly advised to set `autoDeploy: false`" in the Blueprint specification. This prevents code commits from triggering automatic deployments across all instances deployed through the button.

53. user (2026-06-21T14:02:57.804Z)

Web search results for query: "WASB-SBDT nttcom badminton shuttlecock detection license pretrained weights commercial use"

Links: [{"title": "(PDF) Widely Applicable Strong Baseline for Sports Ball Detection and Tracking", "url": "https://www.researchgate.net/publication/376208967_Widely_Applicable_Strong_Baseline_for_Sports_Ball_Detection_and_Tracking"}, {"title": "WASB-SBDT/GET_STARTED.md at main · nttcom/WASB-SBDT", "url": "https://github.com/nttcom/WASB-SBDT/blob/main/GET_STARTED.md"}, {"title": "Sports Ball Detection And Tracking", "url": "https://www.catalyzex.com/s/Sports%20Ball%20Detection%20And%20Tracking"}, {"title": "GitHub - nttcom/WASB-SBDT: [BMVC2023] Widely Applicable Strong Baseline for Sports Ball Detection and Tracking", "url": "https://github.com/nttcom/wasb-"}]


```
sbdT"}, {"title": "One-Shot Badminton Shuttle Detection for Mobile Robots", "url": "https://arxiv.org/pdf/2603.06691"}, {"title": "Automated Service Height Fault Detection Using Computer Vision and Machine Learning for Badminton Matches", "url": "https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10747833/"}, {"title": "Automatic Shuttlecock Fall Detection System in or out of a Court in Badminton Games? Challenges, Problems, and Solutions from a Practical Point of View - PMC", "url": "https://pmc.ncbi.nlm.nih.gov/articles/PMC9655598/"}]
```

Based on the search results, I found information about WASB-SBDT and badminton shuttlecock detection:

WASB-SBDT Overview

WASB is a Sports Ball Detection and Tracking (SBDT) baseline that can be applied to a wide range of sports categories including badminton. The method serves as a Widely Applicable Strong Baseline (WASB) of SBDT, and datasets and codes are available at <https://github.com/nttcom/WASB-SBDT>.

Pretrained Weights

The repository contains evaluation codes of pretrained WASB models, with pretrained weights available for badminton detection (`wasb_badminton_best.pth.tar`) along with models for soccer, tennis, volleyball, and basketball.

License and Commercial Use

Unfortunately, the search results do not contain specific information about the license terms or commercial use restrictions for the WASB-SBDT pretrained weights and model. To find this information, you would need to check the repository's license file or documentation directly on the GitHub repository at <https://github.com/nttcom/WASB-SBDT>.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

54. assistant (2026-06-21T14:03:01.949Z)

Claim 9 (Deploy to Render Blueprint, `autoDeploy:false` advice, provisions on the user's account) is confirmed by the official Render docs. Let me verify the WASB license directly from the repo (Claim 12 is about provenance of weights being uncleared for commercial sharing ? the code itself documents this, which I already saw in the Dockerfile, but let me cross-check the actual repo license) and verify the DPDP Act facts (Claim 13).

55. user (2026-06-21T14:03:07.022Z)

License and Weights Information

****Code License:**** The repository is licensed under the MIT License, as indicated by the badge: "License: MIT"

****Pretrained Weights:**** The page provides no explicit statement about the license or provenance of pretrained models (such as `"wasb_badminton_best"`). While the repository references a `MODEL_ZOO.md` file that likely contains details about available weights, the specific terms governing those models are not included in the provided content.

****Dataset Information:**** No statements regarding academic datasets, usage restrictions, or dataset terms are present in the materials shown above.

To obtain complete information about pretrained model licensing and dataset restrictions, you would need to consult the `MODEL_ZOO.md` file and the full paper referenced at arxiv.org/abs/2311.05237.

56. user (2026-06-21T14:03:15.237Z)

Web search results for query: "India DPDP Act 2023 children under 18 verifiable parental consent data deletion business purpose cross-border transfer restricted countries 2026 enforcement"

Links: [{"title": "Rule 10 of Digital Personal Data Protection Act, 2023 DPDP Rules 2025", "url": "https://www.dpdpa.com/dpdparules/rule10.html"}, {"title": "Transforming data privacy: DPDP Act, 2023 and DPDP Rules, 2025", "url": "https://www.ey.com/en_in/insights/cybersecurity/transforming-data-privacy-digital-personal-data-protection-rules-2025"}, {"title": "Child Data Protection Rules 2025 Under India's DPDP Act | Assurtiv", "url": "https://assurtiv.com/child-data-protection-rules-2025-under-dpdp-act/"}, {"title": "Parental Consent: DPDP Law on Children's Data", "url": "https://www.consent.in/blog/child-consent"}, {"title": "Children's Data Under the DPDP Act, 2023 and DPDP Rules, 2025: The New Compliance Frontier for EdTech, Gaming, Social Media, and Consumer Platforms", "url": "https://ksandk.com/data-protection-and-data-privacy/childrens-data-protection-under-indias-dpdp-rules/"}, {"title": "Navigating Children's Privacy and Parental Consent Under The DPDP Act 2023 - The Quantum Hub", "url": "https://thequantumhub.com/navigating-childrens-privacy-and-parental-consent-under-the-dpdp-act-2023/"}, {"title": "Digital Personal Data Protection Act, 2023 DPDP SECTION 9 WITH INTERPRETATION", "url": "https://www.dpdpa.com/dpdpa2023/chapter-2/section9.html"}, {"title": "Child Data Protection Under DPDP Act: Parental Consent Rules", "url": "https://ksandk.com/data-protection-and-data-privacy/child-data-protection-under-dpdp-act-parental-consent-rules/"}, {"title": "DPDP Act 2023 and the Issue of Parental Consent", "url": "https://www.drishtiiias.com/daily-updates/daily-news-analysis/dpdp-act-2023-and-the-issue-of-parental-consent"}, {"title": "DPDP Rules and the Future of Child Data Safety", "url": "https://www.orfonline.org/english/expert-speak/dpdp-rules-and-the-future-of-child-data-safety"}]

Links: [{"title": "DPDP compliance made practical: A CISO's guide to lawful cross-border data strategy", "url": "https://www.manageengine.com/log-management/cyber-security/dpdp-act-cross-border-data-transfer.html"}, {"title": "Cross-Border Data Transfer under DPDP Act | TCISA", "url": "https://www.tcisa.in/frameworks/dpdp/cross-border-transfer"}, {"title": "DPDP Rules for Cross Border Data Transfers", "url": "https://kalpsystems.com/dpdp/cross-border-data-transfers-dpdp/"}, {"title": "Cross-Border Data Transfers: Reconciling Indian Law with Global Privacy Regimes", "url": "https://www.dpo-india.com/Blogs/cross-border-data-transfers/"}, {"title": "Cross-Border Data Transfers Under DPDP Act 2023: India's Balanced Approach 2026 | Advocate Ritik Khurana", "url": "https://rkhurana.llmadvocates.com/articles/cross-border-data.html"}, {"title": "Cross-Border Data Transfers Under the DPDP Act", "url": "https://elplaw.in/wp-content/uploads/2025/12/Cross-border-data-transfers-under-the-DPDP-Act-what-businesses-need-to-know.pdf"}, {"title": "Data Localisation and Cross-Border Data Transfers Under DPDP Act - Lexology", "url": "https://www.lexology.com/library/detail.aspx?g=7bdb68c9-5e94-4c9d-8b81-9aa7a523d2eb"}, {"title": "From Localisation Debates To A Negative List: Making Cross Border Data Transfers Work Under India's DPDP Act - Data Protection - India", "url": "https://www.mondaq.com/india/data-protection/1764976/from-localisation-debates-to-a-negative-list-making-cross-border-data-transfers-work-under-indias-dpdp-act"}, {"title": "International Data Transfer | India | Global Data and Cyber Handbook | Baker McKenzie Resource Hub", "url": "https://resourcehub.bakermckenzie.com/en/resources/global-data-and-cyber-handbook/asia-pacific/india/topics/international-data-transfer"}, {"title": "Cross-Border Data Transfers After DPDP: A Practical Guide", "url": "https://www.digio.in/blog/cross-border-data-transfers-after-dpdp-a-practical-guide-2/"}]

Links: [{"title": "India DPDP Act implementation: what you need to know for ...", "url": "https://responsibleailabs.ai/knowledge-hub/articles/india-dpdp-act-2026-2027"}, {"title": "Digital Personal Data Protection Act India: Compliance Guide 2026", "url": "https://www.atlassystems.com/blog/digital-personal-data-protection-act-india"}, {"title": "Children's Data Under the DPDP Act, 2023 and DPDP Rules, 2025: The New Compliance Frontier for EdTech, Gaming, Social Media, and Consumer Platforms", "url": "https://ksandk.com/data-protection-and-data-privacy/childrens-data-protection-under-indias-dpdp-rules/"}, {"title": "Parental Consent under the DPDP Act", "url": "https://www.consent.in/blog/child-consent"}, {"title": "Child Data Protection Under DPDP Act: Parental Consent Rules", "url": "https://ksandk.com/data-protection-and-data-privacy/child-data-protection-under-dpdp-act-parental-consent-rules/"}, {"title": "India's DPDP Timeline: Critical Compliance Deadlines for 2026-27", "url": "https://www.india-briefing.com/news/india-dpdp-compliance-timeline-enforcement-2026-27-44740.html"}, {"title": "India Digital Personal Data Protection Act (DPDPA

2025): Updated Guide - CookieYes", "url": "https://www.cookieyes.com/blog/india-digital-personal-data-protection-act-dpdpa/"}], [{"title": "What Is India's Digital Personal Data Protection (DPDP) Act?", "url": "https://www.fortra.com/blog/what-indias-digital-personal-data-protection-act"}, {"title": "Protecting Children's Data in the Digital Age: India's Legal Framework and Policy Imperatives", "url": "https://www.dpo-india.com/Blogs/protecting-child-data/"}], [{"title": "DPDPA - Your Guide to India's Digital Personal Data Protection Act and DPDP Rules, Articles, Blogs, Templates, policies, posters, compliance, case laws on dpdpa", "url": "https://www.dpdpa.com/"}]]

I've completed the web search on India's DPDP Act 2023 regarding children under 18, verifiable parental consent, data deletion, business purposes, and cross-border transfers. Here's a summary of the key findings:

Children Under 18 - Verifiable Parental Consent

Data Fiduciaries must, before processing any personal data of a child or person with disability, obtain verifiable consent from the parent or lawful guardian of such child or person with disability. The DPDP Act defines a child as a person who has not completed 18 years of age. This is broader than several global privacy regimes (such as COPPA in the US which defines a child as under 13)

Online platforms must check the age and identity of the individual identifying as a parent when obtaining verifiable parental consent for processing the data of an under-18 user.

Data Deletion

Individuals must be notified at least 48 hours before erasure. Special classes of data fiduciaries (e.g., e-commerce platforms with 2 crore users, social media intermediaries, gaming platforms) must delete personal data within three years of last interaction.

Business Purpose Restrictions

There is strict prohibition on behavioral tracking, profiling, and targeted advertising directed at children. This is intended to prevent exploitation and manipulation of minors. Parental consent is required unless the processing relates to essential services such as healthcare, education or real time safety.

Cross-Border Data Transfer

Section 16 of the DPDP Act 2023 adopts a negative-list (blacklist) approach: a data fiduciary may transfer personal data to any country or territory except those the Central Government specifically notifies as restricted. As of mid-2026 no such restricted-country list has been notified.

2026 Enforcement Status

Most obligations for Data Fiduciaries, including verifiable parental consent, security safeguards, breach notifications, data minimisation, and processing restrictions for children's data, come into force after 18 months. "Soft enforcement" (guidance and warnings) is expected through 2026, with May 13-14, 2027, widely regarded as the "hard enforcement" date. This marks the end of the 18-month transition period.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

57. assistant (2026-06-21T14:03:25.993Z)

Important findings on Claim 13 (DPDP):

- "treats under-18 as children needing verifiable parental consent" ? **confirmed** (DPDP defines child as under 18; verifiable parental consent required).
- "permits cross-border transfer except to restricted countries" (negative list) ? **confirmed**

(Section 16 negative-list approach).

- "requires deletion once business purpose ends" ? **confirmed** (storage limitation / erasure obligations).
- "consent the primary lawful basis" ? **confirmed** (consent + limited legitimate uses).
- BUT: "enforcement phase 2026" ? this is **partially inaccurate**. The search indicates DPDP Rules 2025 were notified, with a phased transition; "soft enforcement" through 2026, but **hard enforcement** is widely placed at **May 2027** (end of the 18-month transition). So "enforcement phase 2026" is at best loose ? the substantive obligations come into force ~2027. The claim is rated medium confidence by the author, which is appropriate, but the "2026" enforcement framing is imprecise.

Claim 12 (WASB): The code repo is MIT (code), but the search confirms **no explicit license statement on the pretrained weights** ? the provenance for commercial sharing is genuinely unclear. This supports the claim. The repo's `MODEL\_ZOO.md` would have detail ? but the key point (weights provenance not cleared, MIT covers code only) holds. The in-repo Dockerfile documents exactly this (WASB-C3), and weights are NOT baked in / staged via \$WASB_WEIGHTS ? I verified that directly in the Dockerfile (lines 14-16, 53-56). Claim 12 confirmed.

Now let me verify the remaining external claim ? the twelve-factor build-once/deploy-many pattern (Claim 14) ? and double-check the Cloudflare "6 weeks" rotation figure with a more targeted search since it's the weakest external claim.

58. user (2026-06-21T14:03:30.223Z)

Web search results for query: "Cloudflare Access JWT signing key rotation interval "6 weeks" OR "six weeks" certs rotate"

Links: [{"title": "Validate JWTs · Cloudflare One docs", "url": "https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/validating-json/"}, {"title": "Rotate ACM certificate packs · Cloudflare SSL/TLS docs", "url": "https://developers.cloudflare.com/ssl/reference/certificate-rotation/"}, {"title": "Configure the Worker for JWT validation · Cloudflare API Shield docs", "url": "https://developers.cloudflare.com/api-shield/security/jwt-validation/jwt-worker/"}, {"title": "Access Token Signing key rotation", "url": "https://supertokens.com/docs/post-authentication/session-management/security/jwt-signing-key-rotation"}, {"title": "Rotate Signing Keys - Auth0 Docs", "url": "https://auth0.com/docs/get-started/tenant-settings/signing-keys/rotate-signing-keys"}, {"title": "How do I handle JWT key rotation in microservices architectures? | CIAM Q&A - Your Portal for Customer Identity and Access Management Insights", "url": "https://mojoauth.com/ciam-qna/how-to-handle-jwt-key-rotation-in-microservices-architectures"}, {"title": "Access Token Signing key rotation | SuperTokens Docs", "url": "https://supertokens.com/docs/thirdparty/common-customizations/sessions/jwt-signing-key-rotation"}, {"title": "Bad JWT after Access keys were rotated · Issue #28 · cloudflare/cloudflare-access-for-atlassian", "url": "https://github.com/cloudflare/cloudflare-access-for-atlassian/issues/28"}, {"title": "Key Rotation | Okta Developer", "url": "https://developer.okta.com/docs/concepts/key-rotation/"}]

Based on the search results, I found the information you're looking for:

By default, Access rotates the signing key every 6 weeks. Previous keys remain valid for 7 days after rotation to allow time for you to make the update.

This information comes from Cloudflare's official documentation on validating JWTs for Cloudflare Access.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

59. user (2026-06-21T14:03:33.383Z)

Twelve-Factor Config Management

Yes, the twelve-factor methodology strongly advocates this approach. Here are the key statements:

****On strict separation:****

> "Config varies substantially across deploys, code does not."

The methodology emphasizes that "the codebase could be made open source at any moment, without compromising any credentials."

****On environment variables:****

> "The twelve-factor app stores config in environment variables (often shortened to env vars or env)."

The rationale is that env vars are "easy to change between deploys without changing any code" and represent "a language- and OS-agnostic standard."

****On deploy flexibility:****

The document notes that environment variables enable configuration changes across different deployment contexts (staging, production, developer environments) independently, supporting the "build once, deploy many" model through granular, orthogonal controls rather than grouped "environment" configurations.

This design pattern allows the same compiled application to function across multiple deployment targets by simply adjusting its configuration parameters.

60. assistant (2026-06-21T14:03:42.367Z)

Both confirmed:

- ****Claim 6****: The "6 weeks" rotation figure is now ****confirmed**** ? Cloudflare's official docs state "By default, Access rotates the signing key every 6 weeks. Previous keys remain valid for 7 days after rotation." So the "~every 6 weeks" claim is accurate. (The implementation's 10-min JWKS TTL comfortably covers this.)

- ****Claim 14****: Twelve-factor config (build-once-deploy-many, config in env, swap backing service via config only) is fully confirmed by 12factor.net itself. The claim cited an AWS blog as source, but the underlying principle is correct and canonically documented.

I have now verified all 15 claims against code and external sources. Let me do one final cross-check on the exact line numbers cited in Claim 1 (`\_\_init\_\_.py:42-49`) and Claim 4 (`main.py:51,735`) to confirm precision, and confirm no app-layer auth exists in main.py by default (the `resolve\_tenant` is default-OFF). I already saw main.py:51 (allow_origins) and 735 (host=127.0.0.1) ? both exact. The `get\_compute\_backend` logic spans lines 47-49 (claim says 42-49; the function/docstring spans 38-49, the actual switch is 47-49). Close enough ? the core seam claim is accurate.

Let me confirm there's no router-level auth dependency wired into main.py that would contradict Claim 4's "zero app-layer auth" (the auth.py resolver could be invoked somewhere).

61. user (2026-06-21T14:03:43.339Z)

```
backend\main.py:43:from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)
```

```
backend\main.py:291:    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
```

```
backend\main.py:294:        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
```

```
backend\main.py:295:    except edge_auth.EdgeAuthError as e:
```

62. assistant (2026-06-21T14:03:49.411Z)

So edge auth IS wired into main.py (line 294), but it's default-OFF via `resolve\_tenant` returning `None` when `edge\_auth\_enabled=False`. This means Claim 4's "zero app-layer auth" is true ****only in the default-OFF state**** ? when edge auth is enabled, the app DOES verify the JWT signature in-app (not purely an edge gate). This is a meaningful nuance: the claim's framing ("the EDGE gate, not the app, is the tenancy boundary") is partially contradicted by the M9 implementation, where the app itself verifies the Cf-Access JWT signature (per D9's explicit anti-spoof requirement). The claim is accurate for the default/local posture but understates that the app does in-app JWT verification once enabled.

Let me read lines 290-300 of main.py to confirm the exact wiring.

63. user (2026-06-21T14:03:51.070Z)

```
288         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
289         """
290         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
291         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
292         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
293         try:
294             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295         except edge_auth.EdgeAuthError as e:
296             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
297
298         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299         try:
300             validate_input_path(req.video_path, allowed_root=allowed_root)
301             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
```

64. assistant (2026-06-21T14:03:56.166Z)

Confirmed. The edge-auth is wired into the `/api/process` path (line 294), default-OFF, with in-app JWT verification when enabled. This is the nuance for Claim 4 and a point that strengthens Claim 5/6 (D9 anti-spoof via in-app signature verification).

I now have everything verified. Let me compile the structured output.

65. user (2026-06-21T14:05:15.101Z)

Structured output provided successfully

66. assistant (2026-06-21T14:05:19.824Z)

I have completed the adversarial verification of all 14 load-bearing claims. Summary: 12 confirmed, 2 uncertain (Claims 4 and 13). The structured output has been submitted with per-claim verdicts, reasoning, and corrections.